

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\ a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a4947d1c3706cfa61.jsonl`  
- SHA-256: `fa494f5b8501c99efb103727d14fdadd74bc5377dc2c4928cd7d1faa285a9ea9`  
- Source modified: `2026-05-31T04:19:06+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:18:30.239Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://learn.microsoft.com/en-us/azure/foundry/responsible-ai/document-intelligence/data-privacy-security>

****Title:**** Data, privacy, and security for Azure AI Document Intelligence (Microsoft Learn)

****Found via:**** Approval-gated paid/cloud fallbacks & data terms (cost/tradeoffs) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:32.798Z)

I'll fetch the Microsoft Learn page on Azure AI Document Intelligence data privacy and security, then extract the key claims relevant to the research question.

3. assistant (2026-05-31T04:18:35.359Z)

I'll fetch the Azure AI Document Intelligence data privacy and security page.

4. user (2026-05-31T04:18:39.407Z)

layout: Conceptual

title: Data, privacy, and security for Document Intelligence - Foundry Tools | Microsoft Learn

canonicalUrl: <https://learn.microsoft.com/en-us/azure/foundry/responsible-ai/document-intelligence/data-privacy-security>

breadcrumb_path: ../../../../breadcrumb/azure-ai/toc.json

feedback_help_link_url: <https://learn.microsoft.com/answers/tags/133/azure>

feedback_help_link_type: get-help-at-qna

feedback_product_url:

<https://feedback.azure.com/d365community/forum/79b1327d-d925-ec11-b6e6-000d3a4f06a4>

feedback_system: Standard

permissioned-type: public

```
recommendations: true
recommendation_types:
- Training
- Certification
uhfHeaderId: azure-ai-foundry
ms.suite: office
learn_banner_products:
- azure
ms.collection: ce-skilling-ai-copilot
ms.update-cycle: 1825-days
ms.service: azure-ai-document-intelligence
description: This document details issues for data, privacy, and security for Document
Intelligence.
author: laujan
ms.author: lajanuar
manager: nitinme
ms.topic: concept-article
ms.date: 2025-10-15T00:00:00.0000000Z
locale: en-us
document_id: 1e9654b3-a11e-a12b-5bed-06099b30e751
document_version_independent_id: 49824509-c8b8-23f0-e275-0e67417872f1
updated_at: 2026-02-27T23:08:00.0000000Z
original_content_git_url: https://github.com/MicrosoftDocs/azure-ai-docs-
pr/blob/live/articles/foundry/responsible-ai/document-intelligence/data-privacy-security.md
gitcommit: https://github.com/MicrosoftDocs/azure-ai-docs-
pr/blob/93af66dcbf2cece87bb091e8e1549aa1402f9595/articles/foundry/responsible-ai/document-
intelligence/data-privacy-security.md
git_commit_id: 93af66dcbf2cece87bb091e8e1549aa1402f9595
site_name: Docs
depot_name: Learn.azure-ai
page_type: conceptual
toc_rel: ../../toc.json
word_count: 507
asset_id: foundry/responsible-ai/document-intelligence/data-privacy-security
moniker_range_name:
monikers: []
item_type: Content
source_path: articles/foundry/responsible-ai/document-intelligence/data-privacy-security.md
cmProducts:
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/91df3d29-834f-4393-92e2-5e393da1897c
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/0ddf7bfe-6ffa-414f-8b42-372ac6c8be85
spProducts:
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/805a3516-ebe7-4153-baf7-2b9dfcf7b866
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/f90da448-a7e6-4f23-b0d2-c8b100a1e8dc
platformId: 3233d899-4616-7394-83c2-1d87d204d150
---
```

Data, privacy, and security for Document Intelligence - Foundry Tools | Microsoft Learn

Important

Non-English translations are provided for convenience only. Please consult the [‘EN-US’ version of this document](/en-us/azure/ai-foundry/responsible-ai/openai/customer-copyright-commitment) for the definitive version.

This article provides details regarding how Document Intelligence processes your data. Document Intelligence is designed with compliance, privacy, and security in mind. However, you are responsible for its use and the implementation of this technology. It's your responsibility to comply with all applicable laws and regulations in your jurisdiction.

How does Document Intelligence process data?

Authenticate (with subscription or API keys)

The most common way to authenticate access to Document Intelligence is by using the customer's Document Intelligence API key. Each request to the service URL must include an authentication header. This header passes along an API key (or token if applicable), which is used to validate your subscription for a service or group of services. For more information, see [Authenticate requests to Foundry Tools](/en-us/azure/ai-services/authentication?tabs=powershell).

Secure data in transit (for scanning)

All Foundry Tools endpoints, including the Document Intelligence API URLs, use HTTPS URLs for encrypting data during transit. The client operating system needs to support Transport Layer Security (TLS) 1.3 for calling the endpoints. For more information, see [Foundry Tools security](/en-us/azure/security/fundamentals/double-encryption).

Encrypts input data for processing

The incoming data is processed in the same region where the Document Intelligence resource was created. When you submit your documents to a Document Intelligence operation, it starts the process of analyzing the document to extract all text and identify structure and key values in a document. Your data and results are then temporarily encrypted and stored in Azure Storage.

Retrieve the results

The "Get Analyze Results" operation is authenticated against the same API key that was used to call the "Analyze" operation to ensure no other customer can access your data. It returns the analysis job completion status. When the status shows as completed, the operation also returns the extracted results in JSON format.

Data stored by Document Intelligence

****For all analysis**:** To facilitate asynchronous analysis and checking the completion status and returning the extracted results to the customer upon completion, the data and extracted results are stored temporarily in Azure Storage in the same region. All customers in the same region share the temporary storage. The customer's data is logically isolated from other customers with their Azure subscription and API credentials.

****For customer trained models**:** The Custom model feature allows customers to build custom models from training data stored in customer's Azure blob storage locations. The interim outputs after analysis and labeling are stored in the same location. The trained custom models are stored in Azure storage in the same region and logically isolated with their Azure subscription and API credentials.

****Deletes data**:** Analyze response is stored for 24 hours from when the operation completes for retrieval. Customers can delete the analysis response at any time by utilizing the [**Delete Analyze Result**](/en-us/rest/api/aiservices/document-models/delete-analyze-result?view=rest-aiservices-v4.0%20%282024-11-30%29&preserve-view=true&tabs=HTTP) API. After successfully retrieving the analysis results, calling the [**Delete Analyze Result**](/en-us/rest/api/aiservices/document-models/delete-analyze-result?view=rest-aiservices-v4.0%20%282024-11-30%29&preserve-view=true&tabs=HTTP) API permanently purges those results. This action applies to all models.

To learn more about privacy and security commitments, see the [Microsoft Trust Center](https://www.microsoft.com/TrustCenter/CloudServices/Azure/default.aspx).

5. assistant (2026-05-31T04:18:46.657Z)

This page fetched successfully and is highly relevant. It's a Microsoft primary/official documentation page on data handling for Azure AI Document Intelligence, directly relevant to the "approval-gated paid/cloud fallback" portion of the research question. Let me extract the key falsifiable claims.

Key findings:

- Page is official Microsoft Learn documentation (primary source from the vendor).
- `ms.date: 2025-10-15`, `updated\_at: 2026-02-27` ? current and well-maintained.
- Data is processed ****in the cloud, in the same region**** as the resource ? no mention of local/disconnected containers on this page (relevant to muneem's local-first hard constraint: this is cloud-only per this doc).
- Data retention: results stored ****24 hours****, can be deleted via API.
- Encryption in transit (HTTPS/TLS 1.3) and temporarily encrypted at rest in Azure Storage.
- Logical isolation per customer; shared temporary storage within a region.

Now I'll return the structured output.

6. user (2026-05-31T04:18:55.385Z)

Structured output provided successfully

7. assistant (2026-05-31T04:19:06.507Z)

Extracted the key claims from the Azure AI Document Intelligence data privacy and security page.

Summary of what I found, relevant to the muneem research question:

Source: `<https://learn.microsoft.com/en-us/azure/foundry/responsible-ai/document-intelligence/data-privacy-security>` ? official Microsoft Learn documentation (primary/vendor source). Dated `ms.date: 2025-10-15`, last updated `2026-02-27`, so current as of the research window.

Most decision-relevant points for muneem's "approval-gated cloud fallback" assessment:

1. Cloud-only on this page (central): Documents are processed in Azure cloud in the resource's region. This page describes no local/on-device path ? confirming Azure Document Intelligence as a cloud service that would violate muneem's "100% local for statement contents" hard constraint unless explicitly approved. (Note: Azure does separately offer disconnected/Docker containers for some Document Intelligence features, but this particular privacy page does not mention them ? a license/deployment-uncertainty flag worth noting if container self-hosting matters.)
2. 24-hour retention, purgeable (central): Analyze responses stored 24 hours; can be deleted immediately via the Delete Analyze Result API. This is a favorable data-handling term if cloud is ever approved.
3. Encryption + isolation (supporting): HTTPS/TLS 1.3 in transit; temporarily encrypted at rest; shared regional storage but logically isolated per Azure subscription/API key.
4. Custom-model training stays in the customer's own blob storage (tangential) ? i.e., not pooled into a Microsoft training corpus.

One caveat on attribution: this page does not contain an explicit "we do not use your data to train our foundation models" sentence ? that claim typically lives in the broader Azure AI Services terms / Trust Center, not here. I did not assert it, since it wasn't on this page.

All findings returned via the StructuredOutput tool call.